



Markdown, README e documentazione di progetto

Obiettivo della lezione

Oggi iniziamo il progetto vero.

Non ci interessa soltanto “mettere file su GitHub”.

L’obiettivo è imparare a costruire una repository che una persona esterna possa aprire e capire senza doverci chiedere spiegazioni.



Dove siamo arrivati

Nelle lezioni precedenti abbiamo visto:

- uso del terminale;
- gestione di file e cartelle;
- comandi base;
- Git;
- GitHub;
- repository locale e remota;
- commit;
- push e pull;
- collaborazione;
- README e licenze in modo introduttivo.

Oggi usiamo tutto questo per iniziare il progetto finale.

Il passaggio importante è questo:

GitHub non serve solo a caricare file.

GitHub serve a presentare, documentare, versionare e rendere comprensibile un progetto.

Perché documentare?

La documentazione serve a ridurre il tempo necessario per capire un progetto.

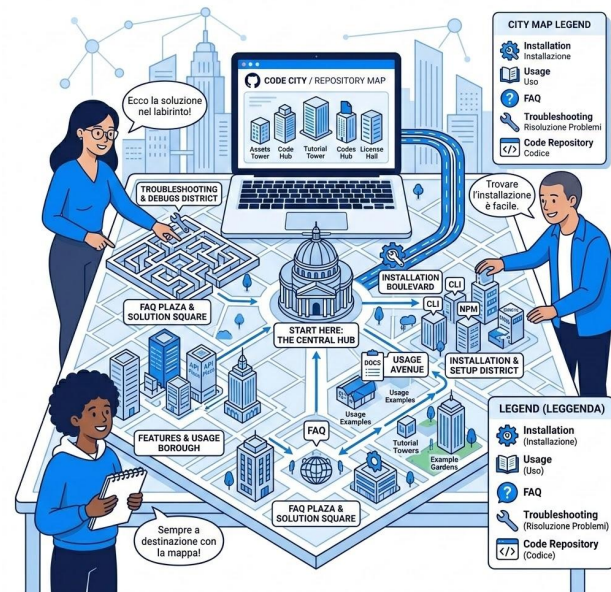
Senza documentazione:

- chi apre la repository non sa da dove iniziare;
- chi deve installare il progetto si blocca;
- chi deve correggere un errore perde tempo;
- chi deve collaborare non capisce la struttura;
- chi valuta il lavoro vede solo una cartella piena di file.

Con una buona documentazione:

- il progetto è più credibile;
- il lavoro del gruppo è più chiaro;
- gli errori si trovano più facilmente;
- l'utente è guidato;
- lo sviluppatore sa dove mettere mano;
- il valutatore capisce meglio il valore del lavoro.

Un README ben fatto è spesso la prima impressione del progetto.





Ripasso operativo Git

Prima di iniziare controlliamo che tutti sappiano fare il ciclo base:

```
git status
git add .
git commit -m "messaggio del commit"
git push
```

Se partiamo da una repository già esistente:

```
git clone URL_REPOSITORY
cd nome-repository
```

Se la repository esiste già sul computer:

```
cd nome-repository
git pull
git status
```



Esercizio – Controllo repository

Passaggi

1. Aprire il terminale.
2. Entrare nella cartella del progetto.
3. Eseguire: `git status`
4. Se la repository è già collegata a GitHub: `git remote -v`
5. Creare un file di prova: `echo "# Nome progetto" > README.md`
6. Salvare il lavoro:

```
git add README.md
git commit -m "aggiunto README iniziale"
git push
```

Output atteso

Il file `README.md` deve comparire nella repository GitHub.

Cos'è Markdown

Markdown è un linguaggio di marcatura leggero.

Serve a scrivere testo formattato usando simboli semplici.

Markdown è molto usato perché:

- è leggibile anche senza essere convertito;
- è facile da scrivere;
- funziona bene con Git;
- è usato in README, documentazione, wiki, issue e pull request;
- può essere trasformato in HTML o PDF.
- Usato anche in JupiterNotebook, LLM output format, etc...

GitHub usa Markdown nei README, nelle issue, nelle pull request e in altri spazi della piattaforma.

GitHub usa GitHub Flavored Markdown, una variante di Markdown basata su CommonMark con estensioni utili come tabelle, task list e altre funzionalità specifiche.



Titolo principale

Questo è un paragrafo.

Lista

- Primo elemento
- Secondo elemento
- Terzo elemento

****Testo in grassetto****

[Testo del link](<https://example.com>)

Markdown non è un formato grafico

scrivere struttura, non decorazione

Markdown non nasce per controllare ogni dettaglio grafico.

Markdown serve a dire:

- questo è un titolo;
- questa è una lista;
- questo è codice;
- questo è un link;
- questa è un'immagine;
- questa è una tabella;
- questa è una citazione;
- questa è una checklist.

Il vantaggio è che il testo rimane semplice.
Una versione dell'HTML ma più semplice

Installazione

Per installare il progetto eseguire:

```
```bash
npm install
```
```

Poi avviare il progetto con:

```
npm start
```

Installazione

Per installare il progetto eseguire:

```
npm install
```

Poi avviare il progetto con:

```
npm start
```

Questo testo è leggibile sia su
GitHub sia nel file originale.

Titoli in Markdown

I titoli si scrivono con il simbolo `#`.

Regola pratica:

- `#` si usa per il titolo principale del documento;
- `##` per le sezioni principali;
- `###` per sottosezioni;
- evitare di usare troppi livelli se non serve.
- Si può arrivare a 6 sottosezioni (#####)

Errore comune

- Usare i titoli solo per “fare il testo grande”.
- I titoli devono rappresentare la struttura logica del documento.
-



`# Titolo livello 1`

`## Titolo livello 2`

`### Titolo livello 3`

`#### Titolo livello 4`



`# Meteo App`

`## Obiettivo`

`## Requisiti`

`## Installazione`

`## Utilizzo`

`## Struttura del progetto`

`## Autori`

`## Licenza`

Esercizio 2: struttura titoli

Esercizio 2: struttura titoli

Nel file `README.md`, creare questa struttura:

Per ora non serve completare tutto.

Serve creare lo scheletro corretto.

Commit richiesto

```
git add README.md
```

```
git commit -m "creata struttura README"
```

```
git push
```

```
# Nome progetto
```

```
## Descrizione
```

```
## Obiettivo
```

```
## Funzionalità principali
```

```
## Requisiti
```

```
## Installazione
```

```
## Utilizzo
```

```
## Esempi
```

```
## Struttura del progetto
```

```
## Autori
```

```
## Licenza
```

Paragrafi, grassetto e corsivo

Testo normale, grassetto e corsivo

Un paragrafo in Markdown si scrive normalmente.

- Per separare due paragrafi, lasciare una riga vuota.

Grassetto:

****testo importante****

Corsivo:

testo in evidenza

Grassetto e corsivo:

******testo molto evidenziato******

Usare il grassetto per evidenziare concetti importanti, non per decorare tutta la pagina.



Questo progetto permette di consultare una lista di film.

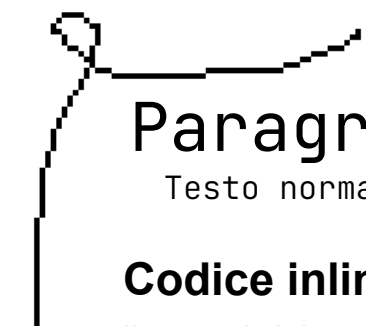
****Attenzione:**** per usare il progetto è necessario un browser moderno.

Nota: la versione attuale è ancora in sviluppo.

Questo progetto permette di consultare una lista di film.

Attenzione: per usare il progetto è necessario un browser moderno.

Nota: la versione attuale è ancora in sviluppo.



Paragrafi, grassetto e corsivo

Testo normale, grassetto e corsivo

Codice inline

Il comando ``git status`` mostra lo stato della repository.

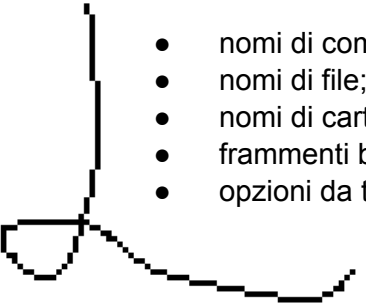
Risultato:

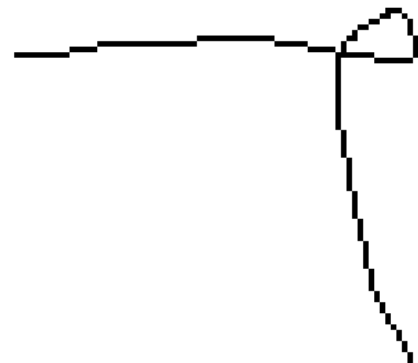
Il comando `git status` mostra lo stato della repository.

Quando usare il codice inline?

il codice inline non serve per “decorare”, serve per distinguere elementi tecnici dal testo normale.

Usarlo per:

- nomi di comandi;
 - nomi di file;
 - nomi di cartelle;
 - frammenti brevi di codice;
 - opzioni da terminale.
- 



Esercizio Inline

- **Trasformare questo testo grezzo:**

Per controllare lo stato della repository usa `git status`. Poi aggiungi i file con `git add .` e salva tutto con `git commit`.



Per controllare lo stato della repository usa ``git status``.

Poi aggiungi i file con ``git add .`` e salva tutto con ``git commit``.

Liste puntate e numerate

Le liste servono a rendere le informazioni più leggibili.

Lista puntata

- Ricerca dati
- Visualizzazione risultati
- Salvataggio preferiti

Lista numerata

1. Aprire il progetto
2. Installare le dipendenze
3. Avviare il programma
4. Aprire il browser

Quando usarle

Usare liste puntate per elementi senza ordine preciso.

Usare liste numerate per procedure passo-passo.

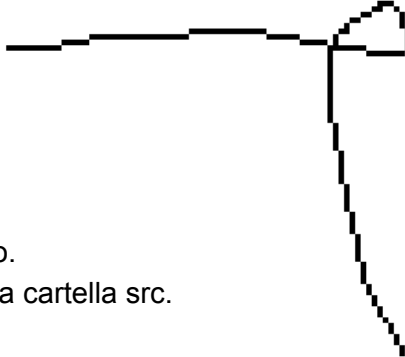
Liste annidate

- Documentazione
 - README
 - Guida installazione
 - Guida utente
- Codice
 - HTML
 - CSS
 - JavaScript



Esercizio

Scrivere in `docs/installazione.md` una procedura di installazione con qualche passaggio numerato.
La guida deve spiegare come poter scaricare una repository e aprire un file `index.html` presente dentro la cartella `src`.





Link in Markdown

I link servono per collegare il documento ad altre risorse.

Sintassi

[Testo visibile](URL)

Esempio

Vai alla [documentazione ufficiale di GitHub](<https://docs.github.com/>).

Link a file interni della repository

Leggi la [guida di installazione](docs/[installazione.md](#)).

Consulta le [FAQ](docs/[faq.md](#)).

Vai alla [guida utente](docs/[guida-utente.md](#)).

Link relativi

I link relativi sono preferibili quando si punta a file interni della repository.

Esempio: [Installazione](docs/[installazione.md](#))

è meglio di: [Installazione](<https://github.com/utente/repo/blob/main/docs/installazione.md>)

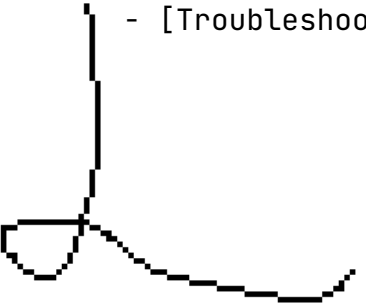
perché funziona meglio se la repository viene copiata o rinominata.



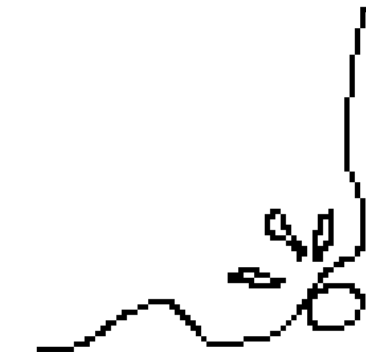
Esercizio

Aggiungere al README una sezione:

Documentazione

- [Installazione](docs/installazione.md)
 - [Guida utente](docs/guida-utente.md)
 - [FAQ](docs/faq.md)
 - [Troubleshooting](docs/troubleshooting.md)
- 

— WWW.DOCUMENT.ME —



Immagini in Markdown

Le immagini rendono la documentazione più chiara.

Sintassi

`![Testo alternativo](percorso/immagine.png)`

Esempio

`![Screenshot della homepage](assets/immagini/homepage.png)`

Testo alternativo

Il testo tra parentesi quadre serve a descrivere l'immagine.

Esempio buono:

`![Schermata iniziale dell'app con barra di ricerca e lista risultati](assets/immagini/homepage.png)`

Esempio scarso:

`![immagine](assets/immagini/homepage.png)`

Si possono inserire anche GIF e VIDEO

Regole pratiche

- Salvare le immagini in `assets/immagini/`.
- Usare nomi file chiari.
- Evitare spazi nei nomi file.
- Usare nomi come `homepage.png`, `errore-login.png`, `risultati-ricerca.png`.
- Non usare nomi come `Screenshot 2026-05-25 alle 12.03.44.png`.



Esercizio

Creare un file fittizio o usare un'immagine reale:

`assets/immagini/homepage.png`

Poi inserirlo nel README.

Blocchi di codice

AKA: Perché non basta “salvare tante copie del file”

Quando dobbiamo mostrare comandi o codice, usiamo i blocchi di codice.

Sintassi

```
```nomelinguaggio  
codice del linguaggio
```
```

Perché specificare il linguaggio?

Scrivere **bash**, **html**, **css**, **js**, **json**, **bash** dopo le tre backtick aiuta l'evidenziazione della sintassi.

GitHub supporta blocchi di codice recintati e formattazione del codice nei file Markdown.

Esempio per HTML

```
```html  
<h1>Nome progetto</h1>
<p>Descrizione del progetto.</p>
```
```

Esempio per JavaScript

```
```js  
const button =
document.querySelector("#searchButton");
button.addEventListener("click", () => {
 console.log("Click");
});
```
```



Esercizio

Scrivere in [docs/installazione.md](#) un blocco di codice con i comandi Git per clonare la repository.



Esercizio - Soluzione

Scrivere in `docs/installazione.md` un blocco di codice con i comandi Git per clonare la repository.

Clonare la repository

Aprire il terminale ed eseguire:

```
```bash
```

```
git clone https://github.com/nomeutente/nome-progetto.git
```

```
cd nome-progetto
```

```
```
```



Tabelle in Markdown

Le tabelle servono per confrontare informazioni in modo ordinato.

Sintassi

```
Nome colonna 1	Nome colonna 2
Riga 1 colonna 1	Riga 1 colonna 2
Riga 2 colonna 1	Riga 2 colonna 2
```

Quando usare una tabella?

Usarla per:

- comandi e significati;
- problemi e soluzioni;
- funzionalità e descrizioni;
- ruoli del gruppo;
- file e scopo;
- requisiti minimi.

| Comando | Significato |
|-------------------------|----------------------------------|
| <code>git status</code> | Mostra lo stato della repository |
| <code>git add .</code> | Aggiunge tutte le modifiche |
| <code>git commit</code> | Salva una versione |
| <code>git push</code> | Invia su GitHub |



Esercizio

Aggiungere al README una tabella “Struttura del progetto”. spiega brevemente cosa sono la cartella docs, src, assets/immagini



Esercizio - soluzione

Aggiungere al README una tabella “Struttura del progetto”.

Struttura del progetto

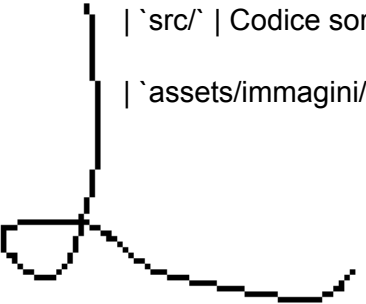
| Percorso Descrizione |
|------------------------|
|------------------------|

| |
|---------|
| --- --- |
|---------|

| |
|---------------------------------------|
| `docs/` Documentazione del progetto |
|---------------------------------------|

| |
|--------------------------|
| `src/` Codice sorgente |
|--------------------------|

| |
|--|
| `assets/immagini/` Immagini e screenshot |
|--|



Citazioni e note importanti

Citazioni e note

Markdown permette di evidenziare parti importanti con le citazioni.

Citazione semplice

> Questo è un testo evidenziato.

Uso nella documentazione tecnica

Le citazioni sono utili per:

- note;
- avvisi;
- suggerimenti;
- limitazioni;
- informazioni importanti.

Alert su GitHub

GitHub supporta anche una sintassi per alert come note, warning, tip e caution.

Esempio:

> [!NOTE]

> Questa guida è pensata per utenti alla prima esperienza.

> [!WARNING]

> Non cancellare la cartella `.git`, altrimenti perdi la storia della repository.

Citazioni e note importanti

Nota: prima di eseguire `git push`, controllare sempre lo stato con `git status`.

Note

Questa guida è pensata per utenti alla prima esperienza.

Warning

Non cancellare la cartella `.git`, altrimenti perdi la storia della repository.



Checklist in Markdown

Le checklist servono per monitorare attività.

Le checklist sono supportate da GitHub Flavored Markdown, soprattutto in **issue**, **pull request**

Sintassi

- [x] Repository creata
- [x] README aggiunto
- [] Guida installazione completata
- [] FAQ completata
- [] Screenshot aggiunti

Quando usarle?

Le checklist sono utili per:

- cose da fare;
- stato del progetto
- roadmap



Indice manuale

Nei documenti lunghi è utile inserire un indice.

Esempio

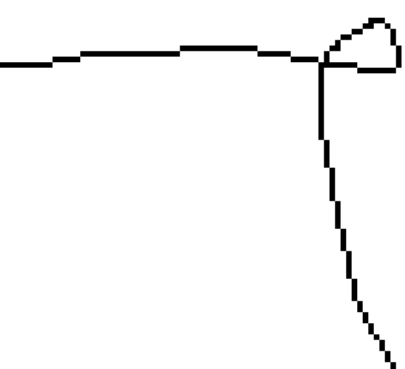
README

Indice

- [Descrizione](#descrizione)
- [Funzionalità](#funzionalità)
- [Installazione](#installazione)
- [Utilizzo](#utilizzo)
- [Struttura del progetto](#struttura-del-progetto)
- [Licenza](#licenza)

Descrizione

Testo...



Regola pratica

Nei README brevi non serve un indice.

Nei README lunghi aiuta molto.

GitHub supporta section links e tabelle dei contenuti/ancore nei file Markdown.

Come funzionano i link interni?

Il link:

[Struttura del progetto](#struttura-del-progetto)

punta al titolo:

Struttura del progetto

Mermaid: diagrammi dentro Markdown

Mermaid permette di scrivere diagrammi usando testo.

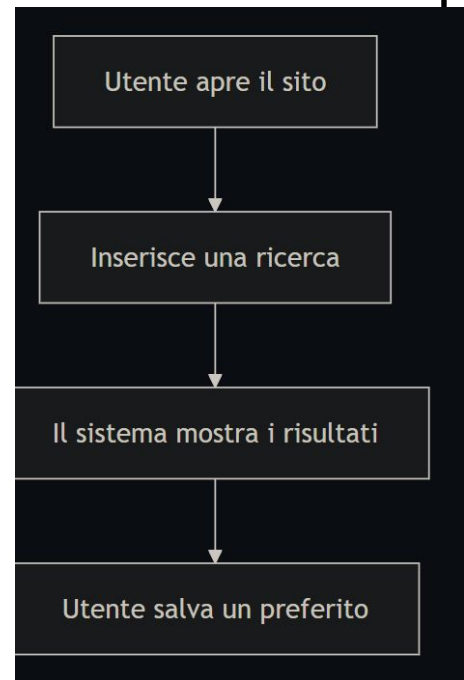
È utile perché:

- il diagramma resta dentro la documentazione;
- può essere versionato con Git;
- si modifica come normale testo;
- evita di dover rifare immagini a mano.

GitHub supporta diagrammi Mermaid nei file Markdown, issue, pull request, discussioni e wiki.

Esempio: flowchart

```
```mermaid
flowchart TD
 A[Utente apre il sito] --> B[Inserisce una ricerca]
 B --> C[Il sistema mostra i risultati]
 C --> D[Utente salva un preferito]
```
```



Mermaid: diagrammi dentro Markdown

```
```mermaid
```

```
flowchart TD
```

```
A[Utente apre il sito] → B[Visualizza homepage]
```

```
B → C[Inserisce una parola chiave]
```

```
C → D[Il sistema filtra i dati]
```

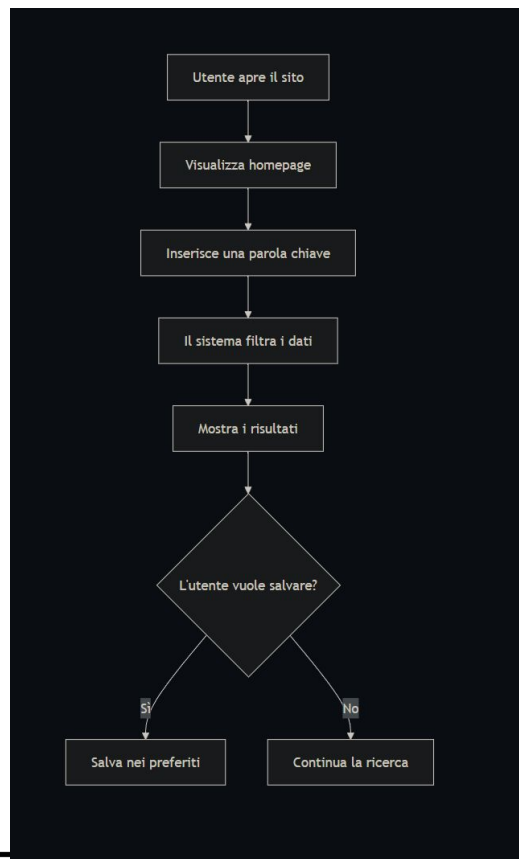
```
D → E[Mostra i risultati]
```

```
E → F{L'utente vuole salvare?}
```

```
F -- Sì --> G[Salva nei preferiti]
```

```
F -- No --> H[Continua la ricerca]
```

```
```
```





Esercizio riepilogo Markdown

Scrivi una pagina Markdown che spiega un comando visto nelle lezioni precedenti.

Scegli uno tra:

- `cd`
- `ls`
- `mkdir`
- `touch`
- `git status`
- `git add`
- `git commit`
- `git push`
- `git pull`
- `git clone`

La pagina deve contenere

- titolo;
- breve descrizione;
- sintassi;
- esempio;
- tabella con almeno 2 casi d'uso;
- errore comune;
- soluzione dell'errore;
- almeno un blocco di codice.

Markdown è lo standard in tutti i docs

Qualche esempio:

- <https://docs.pytorch.org/tutorials/beginner/basics/intro.html>
- <https://docs.astro.build/it/guides/configuring-astro/>
- <https://platform.claude.com/docs/en/api/overview>



Cos'è un README

Il **README.md** è il punto di ingresso della repository.

È il primo file che una persona legge per capire:

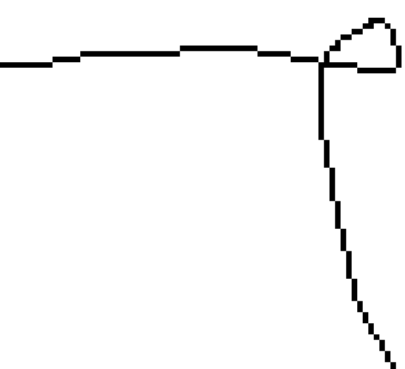
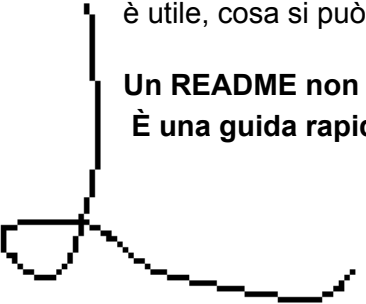
- che cos'è il progetto;
- a cosa serve;
- come usarlo;
- come installarlo;
- dove trovare la documentazione;
- chi lo ha creato;
- quale licenza usa.

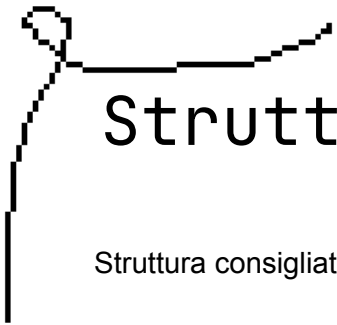
GitHub spiega che il README serve a comunicare perché il progetto è utile, cosa si può fare con il progetto e come usarlo.

Un README non è un tema scolastico.

È una guida rapida per orientarsi nel progetto.

— www.Duca1014.it —





Struttura di un buon README

Struttura consigliata del README ->

Non tutte le sezioni sono sempre obbligatorie

Nome progetto

Breve descrizione del progetto.

Indice

Descrizione

Funzionalità principali

Requisiti

Installazione

Utilizzo

Struttura repository

Documentazione

Screenshot

Roadmap

Autori

Licenza



— www.Docker.io —

Esempio di README incompleto

Problemi

- Il nome è generico.
- Non spiega l'obiettivo.
- Non dice quali funzionalità ci sono.
- Non indica i requisiti.
- Non spiega bene come si usa.
- Non collega la documentazione.
- Non indica autori/licenza.
- Non permette a un esterno di capire il progetto.



Progetto

Questo è il nostro progetto del corso.

Abbiamo fatto una pagina con delle cose.

Per usarlo aprire il file.



Cos'è un README

Il **README.md** è il punto di ingresso della repository.

È il primo file che una persona legge per capire:

- che cos'è il progetto;
- a cosa serve;
- come usarlo;
- come installarlo;
- dove trovare la documentazione;
- chi lo ha creato;
- quale licenza usa.

GitHub spiega che il README serve a comunicare perché il progetto è utile, cosa si può fare con il progetto e come usarlo.

Un README non è un tema scolastico.

È una guida rapida per orientarsi nel progetto.

Esempio di README migliore

Catalogo Film

Catalogo Film è una piccola applicazione web didattica che permette di cercare film, visualizzare risultati e salvare elementi preferiti.

Funzionalità

- Ricerca per titolo.
- Visualizzazione dei risultati.
- Scheda dettaglio del film.
- Salvataggio dei preferiti.
- Documentazione utente nella cartella `docs/`.

Requisiti

- Browser moderno.
- Connessione internet se si usano risorse esterne.
- Git, se si vuole clonare la repository.

Utilizzo

1. Aprire `index.html`.
2. Inserire una parola nella barra di ricerca.
3. Premere il pulsante Cerca.
4. Selezionare un risultato.
5. Salvare il film nei preferiti.

Documentazione

- [\[Guida utente\]\(docs/guida-utente.md\)](#)
- [\[Installazione\]\(docs/installazione.md\)](#)
- [\[FAQ\]\(docs/faq.md\)](#)
- [\[Troubleshooting\]\(docs/troubleshooting.md\)](#)

Autori

Progetto realizzato dal gruppo X per il corso FISM 2026.

Licenza

Questo progetto è distribuito con licenza MIT.

— [www.DOCUMENT.ME](#) —



Scrivere le funzionalità

Una funzionalità deve essere descritta in modo concreto.

Esempio scarso

Funzionalità

- Cerca
- Mostra
- Preferiti

Esempio migliore

Funzionalità

- ****Ricerca dati****: l'utente può cercare elementi inserendo una parola chiave.
- ****Visualizzazione risultati****: i risultati vengono mostrati in una lista leggibile.
- ****Salvataggio preferiti****: l'utente può salvare gli elementi più interessanti.



Installazione e utilizzo

La sezione installazione spiega come preparare il progetto. La sezione utilizzo spiega come usarlo.

Installazione

Installazione

Clonare la repository:

```
```bash
git clone https://github.com/utente/nome-progetto.git
```
```

...

Entrare nella cartella:

```
cd nome-progetto
```



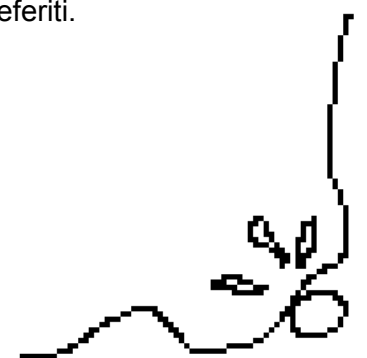
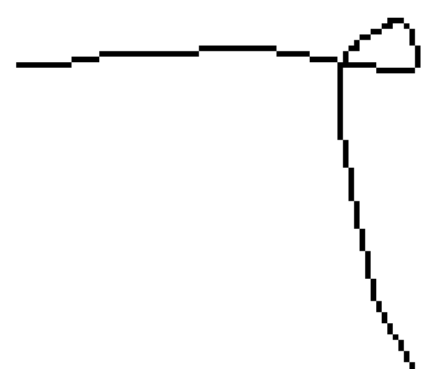
Utilizzo

Utilizzo

1. Aprire il file `index.html` nel browser.
2. Usare la barra di ricerca.
3. Selezionare un risultato.
4. Aggiungere elementi ai preferiti.

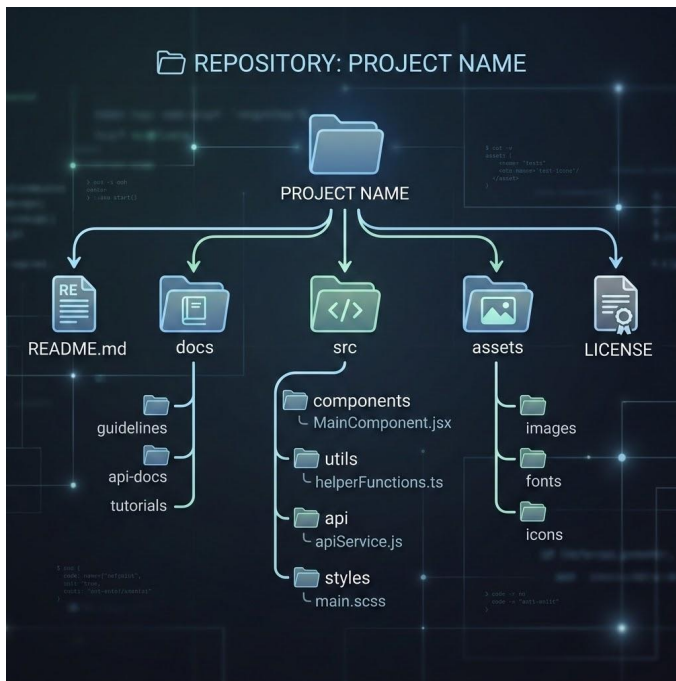
Differenza

| Sezione | Domanda |
|---------------|---------------------------|
| Installazione | Come preparo il progetto? |
| Utilizzo | Come uso il progetto? |



Struttura repository nel README

Nel README bisogna spiegare le cartelle principali. La struttura ad albero è molto utile. Anche progetti piccoli sembrano più seri se sono organizzati.



Struttura repository

```
``text
nome-progetto/
├── README.md
├── LICENSE
├── CHANGELOG.md
├── docs/
│   ├── guida-utente.md
│   ├── installazione.md
│   ├── faq.md
│   └── troubleshooting.md
├── src/
├── assets/
│   └── immagini/
```

Docs as Code

Docs as Code significa trattare la documentazione come il codice.

Quindi la documentazione viene:

- scritta in file testuali;
- salvata nella repository;
- versionata con Git;
- modificata tramite commit;
- revisionata;
- collegata a issue e pull request;
- aggiornata insieme al progetto.



Idea chiave

La documentazione non è un file Word abbandonato alla fine.
È parte del progetto.

Tipi di documentazione software

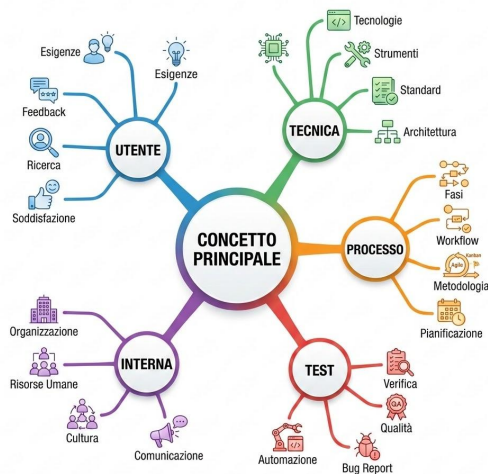
Un progetto può contenere documenti diversi, per pubblici diversi.

Non tutti i documenti devono dire le stesse cose.

Una FAQ non è una guida installazione.

Un README non è un manuale completo.

Una API reference non è una guida utente.



| Tipo | Esempi | A chi serve |
|----------------------------|--|------------------------------|
| Documentazione utente | manuale, guida rapida, FAQ | utenti finali |
| Documentazione tecnica | architettura, struttura, API | sviluppatori |
| Documentazione di processo | installazione, configurazione, runbook | chi deve eseguire il sistema |
| Documentazione di test | checklist, test case, bug report | gruppo, docente, tester |
| Documentazione interna | commenti, changelog, note | team di sviluppo |

Documenti diversi, scopi diversi

Errore comune

Mettere tutto nel README.

Scelta migliore

README breve e collegamenti ai documenti specifici.

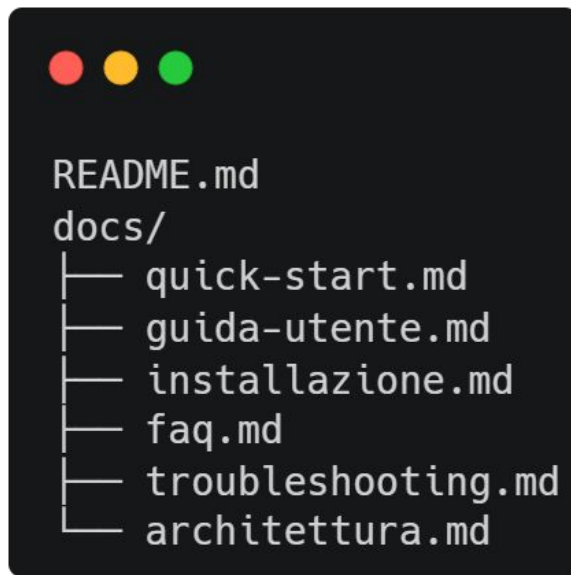
| Documento | Risponde a | Esempio contenuto |
|---------------------------------|------------------------------|--------------------------------|
| <code>README.md</code> | Che cos'è il progetto? | panoramica, funzionalità, link |
| <code>guida-utente.md</code> | Come lo uso? | istruzioni, schermate, esempi |
| <code>installazione.md</code> | Come lo preparo? | requisiti, setup, comandi |
| <code>faq.md</code> | Domande frequenti | dubbi ricorrenti |
| <code>troubleshooting.md</code> | Cosa faccio se non funziona? | problemi e soluzioni |

Information Architecture

Information Architecture significa organizzare le informazioni in modo che siano facili da trovare e capire.

Domande guida:

- Qual è il primo documento che una persona legge?
- Dove metto le istruzioni di installazione?
- Dove metto le FAQ?
- Dove metto i problemi comuni?
- Dove metto gli screenshot?
- Dove metto le informazioni per sviluppatori?



Esercizio – Classificare i documenti

Ogni gruppo deve classificare i propri documenti secondo Diátaxis.

| File | Categoria | Perché |

|---|---|---|

| README.md | | |

| docs/quick-start.md | | |

| docs/installazione.md | | |

| docs/guida-utente.md | | |

| docs/faq.md | | | |

| docs/troubleshooting.md | | | |

| docs/architettura.md | | | |



Esercizio – Classificare i documenti -

Soluzione

Ogni gruppo deve classificare i propri documenti secondo Diátaxis.

| File | Categoria | Perché |

|---|---|---|

| README.md | Panoramica / ingresso | Presenta il progetto |

| docs/quick-start.md | Tutorial | Fa iniziare rapidamente |

| docs/installazione.md | How-to | Spiega come installare |

| docs/guida-utente.md | How-to / Tutorial | Spiega come usare |

| docs/faq.md | Reference leggera | Risponde a domande frequenti |

| docs/troubleshooting.md | How-to | Risolve problemi |

| docs/architettura.md | Explanation | Spiega struttura e scelte |



Target e linguaggio

La stessa funzionalità può essere spiegata in modi diversi.

Per utente non tecnico

Ricerca

Per cercare un elemento, scrivi una parola nella barra di ricerca e premi il pulsante Cerca.

Il sistema mostrerà solo i risultati collegati alla parola inserita.

Per sviluppatore

Ricerca

La ricerca filtra l'array di oggetti in base al valore inserito dall'utente.

Il filtro confronta la query con il campo `title` di ogni elemento.

| Target | Linguaggio |
|--------------------|---|
| utente finale | semplice, operativo, senza dettagli interni |
| sviluppatore | tecnico, preciso, con riferimenti a file e dati |
| docente/valutatore | chiaro, completo, con motivazione delle scelte |

FAQ

Le FAQ rispondono a domande frequenti. (Frequently Asked Questions)

FAQ

Il progetto funziona senza internet?

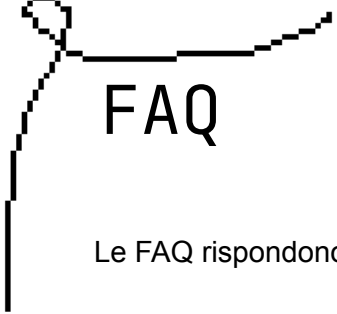
Sì, se tutti i file sono locali e non vengono usate risorse esterne.

Devo installare qualcosa?

No, se il progetto è composto solo da HTML, CSS e JavaScript base.

Dove trovo la guida utente?

La guida utente si trova in ``docs/guida-utente.md``.



FAQ

Le FAQ rispondono a domande frequenti. (Frequently Asked Questions)

Buone FAQ

- rispondono a domande reali;
- sono brevi;
- non duplicano tutta la guida utente;
- rimandano ad altri documenti quando serve.

Troubleshooting

Il troubleshooting aiuta quando qualcosa non funziona.



Troubleshooting

L'immagine non viene visualizzata

Possibile causa

Il percorso dell'immagine è sbagliato.

Soluzione

Controllare che il file esista nella cartella ``assets/immagini/``.

![[[Homepage](#)](assets/immagini/homepage.png)]

Quick start

Una quick start guide serve a far partire l'utente rapidamente.

Non spiega tutto.

Spiega il minimo necessario per iniziare.

| Documento | Scopo |
|---------------|--------------------------------|
| Quick start | partire subito |
| Guida utente | spiegare bene l'uso |
| Installazione | preparare ambiente e requisiti |

Quick Start

1. Scarica il progetto

```
```bash
git clone https://github.com/utente/progetto.git
```
```

2. Apri la cartella

```
```bash
cd progetto
```
```

3. Avvia il progetto

Apri il file `index.html` nel browser.

4. Prova la ricerca

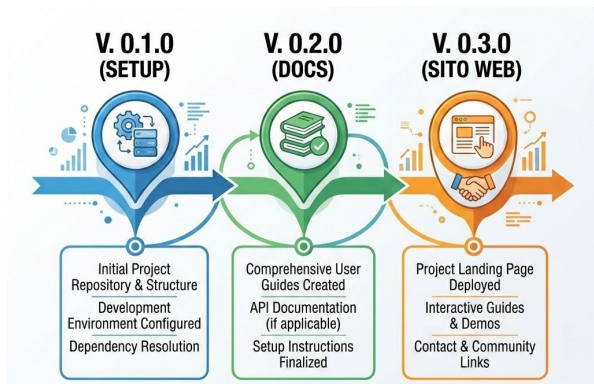
Scrivi una parola nella barra di ricerca e premi Cerca.

CHANGELOG

Il **CHANGELOG.md** registra le modifiche principali del progetto

Perché serve

- rende visibile l'evoluzione del progetto;
- aiuta a ricostruire cosa è cambiato;
- prepara al versionamento serio;
- migliora la valutazione del lavoro..



Changelog

0.1.0 - 2026-05-25

Aggiunto

- Creata struttura iniziale del progetto.
- Aggiunto README.
- Aggiunta cartella `docs/`.
- Aggiunta guida utente iniziale.
- Aggiunta FAQ.

Modificato

- Migliorata struttura repository.

Corretto

- Corretti link interni nel README.

Strumenti e formati per la documentazione

Una bella carrellata di quello che vedremo

| Strumento/formato | A cosa serve | Quanto lo usiamo |
|-----------------------------|-------------------------------------|------------------|
| Markdown | README e docs | molto |
| GitHub/GitLab | versionamento documentazione | molto |
| MkDocs | sito statico da Markdown | demo leggera |
| Docusaurus | documentazione web strutturata | solo panoramica |
| AsciiDoc | documentazione tecnica avanzata | solo citazione |
| reStructuredText/Sphinx | documentazione Python | solo citazione |
| JSDoc | documentazione da codice JavaScript | più avanti |
| Swagger/OpenAPI | documentazione API | più avanti |
| Word/LibreOffice | documenti formali | panoramica |
| Confluence/Notion/MediaWiki | wiki aziendali ↓ | panoramica |

Da Markdown a sito documentale

Con strumenti come MkDocs è possibile trasformare file Markdown in un sito di documentazione.

Esempio di Struttura compatibile con MkDocs

progetto/

├─ mkdocs.yml

└─ docs/

│ └─ index.md

│ └─ installazione.md

│ └─ guida-utente.md

Esempio **mkdocs.yml**

site_name: Documentazione progetto FISM

nav:

- Home: index.md
- Installazione: installazione.md
- Guida utente: [guida-utente.md](#)

Comandi tipici

mkdocs serve

mkdocs build

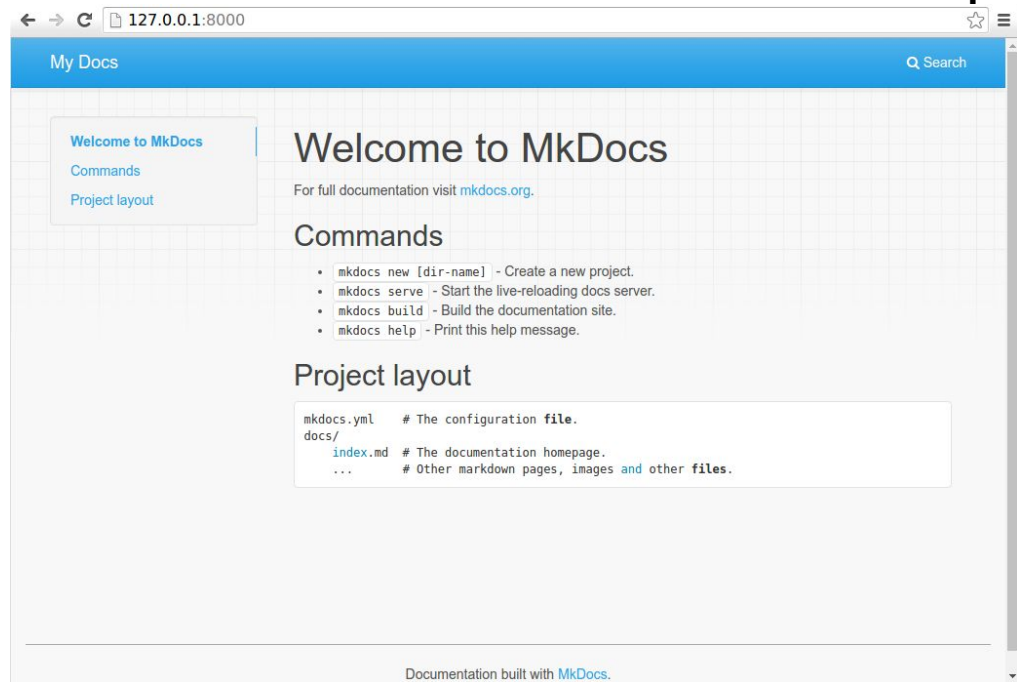
Da Markdown a sito documentale

Per questa giornata

Non è obbligatorio installare MkDocs.

È sufficiente capire il principio:

scrivo Markdown → organizzo file → genero sito.



— www.DOCUMENTA.ME —

Esercizio: documenti nella cartella docs

iniziare questi file:

docs/

- quick-start.md
- guida-utente.md
- installazione.md
- faq.md
- troubleshooting.md

Contenuto minimo

quick-start.md

- 4–5 passi per iniziare subito.

guida-utente.md

- descrizione delle funzionalità;
- esempio d'uso.

installazione.md

- requisiti;
- clonazione repository;
- apertura progetto.

faq.md

- almeno 5 domande.

troubleshooting.md

- almeno 3 problemi con soluzione.

Esercizi avanzati: Markdown reale, GitHub reale

Lo useremo per produrre cose reali:

1. una homepage personale GitHub;
2. un CV scritto in Markdown;
3. una versione web del CV pubblicata con GitHub Pages;
4. una versione PDF generata automaticamente;
5. una collaborazione su repository con errori tramite issue e pull request.

Servono anche per:

- portfolio personale;
- documentazione professionale;
- CV tecnico;
- collaborazione;
- revisione;
- pubblicazione automatica.

Creare la propria homepage GitHub

GitHub permette di mostrare un README direttamente nella pagina profilo personale.

Per farlo bisogna creare una repository speciale:

username/username

La repository deve essere:

- pubblica;
- con lo stesso nome dello username GitHub;
- con un file **README.md** nella root;
- con contenuto dentro il README.

GitHub documenta che il profile README viene mostrato nel profilo se la repository ha lo stesso nome dell'account, è pubblica e contiene un **README.md** nella root.

Y11
XiaomingX · lui/lui
Seguire

Sviluppatore software presso X | Linux, Java, Spring, Python, Golang, Next.js | Appassionato di Open Source | Innovatore LLM | Laurea Magistrale in Ingegneria del Software

11.400 follower · 5.900 persone seguite

jobleap.cn
Giappone
https://www.jobleap.cn
@sedink
https://fujiin.cn/user/2873978147692910
https://mp.jobleap4u.com

Risultati conseguiti

Chi sono

Ciao, sono XiaomingX, un ingegnere della sicurezza che ama esplorare nuovi sistemi e vulnerabilità di sicurezza.

33 Sono profondamente interessato all'apprendimento automatico, agli algoritmi di apprendimento per rinforzo, alla programmazione GPU e alla creazione di plugin frontend e progetti di toolchain.

🔧 Ho lavorato con Python, Node.js e Spring Boot e mi diverto a creare strumenti pratici e a fare esperimenti divertenti.

🔥 Ultimamente sto sviluppando **jobleap.cn**, un sito progettato per avvicinare chi cerca lavoro e i recruiter, riducendo l'asimmetria informativa e contribuendo così a risparmiare tempo, a evitare discrepanze e a rendere più agevole il processo di assunzione. Spero che lo troviate utile!

💖 Sentitevi liberi di inviare pull request per migliorare i miei progetti e renderli migliori.

💡 Puoi contattarmi tramite X o seguimi su GitHub per rimanere aggiornato sui miei ultimi lavori.

💤 Spesso programmo a tarda notte, quindi potrei rispondere con un po' di ritardo, ma mi farebbe piacere ricevere i vostri suggerimenti domattina.

Struttura consigliata del Profile README

Un buon profile README può contenere ->

Deve sembrare un piccolo portfolio, non un albero di Natale.



```
# Ciao, sono Nome Cognome
```

```
Breve descrizione personale.
```

Cosa sto imparando

- HTML
- CSS
- JavaScript
- Git e GitHub
- Documentazione tecnica

Progetti

```
Progetto	Descrizione	Link
Progetto FISM	Repository del progetto finale	link
CV Markdown	CV scritto in Markdown	link
```

Competenze

```
Badge o lista delle tecnologie usate.
```

Contatti

- GitHub: ...
- Email: ...
- LinkedIn: ...

— WWW.DOCUMENTARE —

Badge carini con Shields.io

I badge servono per rendere più leggibile il profilo o il README.

Shields.io permette di creare badge statici o dinamici in formato immagine, spesso usati nei README GitHub. Il servizio si presenta come uno strumento per creare badge concisi, coerenti e leggibili.



```
![HTML](https://img.shields.io/badge/HTML5-base-orange)
![CSS](https://img.shields.io/badge/CSS3-base-blue)
![JavaScript](https://img.shields.io/badge/JavaScript-learning-yellow)
![GitHub](https://img.shields.io/badge/GitHub-documentation-black)
![Markdown](https://img.shields.io/badge/Markdown-README-lightgrey)
```

HTML5 base

CSS3 base

JavaScript learning

GitHub documentation

Markdown README

Badge carini con Shields.io

```
![[HTML5]](https://img.shields.io/badge/HTML5-E34F26?style=for-the-badge&logo=html5&logoColor=white)
![[CSS3]](https://img.shields.io/badge/CSS3-1572B6?style=for-the-badge&logo=css3&logoColor=white)
![[JavaScript]](https://img.shields.io/badge/JavaScript-F7DF1E?style=for-the-badge&logo=javascript&logoColor=black)
![[Git]](https://img.shields.io/badge/Git-F05032?style=for-the-badge&logo=git&logoColor=white)
![[GitHub]](https://img.shields.io/badge/GitHub-181717?style=for-the-badge&logo=github&logoColor=white)
```

I badge devono aiutare la lettura.
Non devono sostituire una
descrizione chiara.

 HTML5

 CSS3

 JAVASCRIPT

 GIT

 GITHUB

— WWW.DIVCRO.ME —

Esempio Profile README

Ciao, sono Mario Rossi

Sto seguendo il corso FISM 2026 e sto imparando a costruire documentazione tecnica, repository GitHub ordinate e piccole pagine web.

Cosa sto imparando

- Scrittura Markdown
- Git e GitHub
- README tecnici
- Documentazione di progetto
- HTML, CSS e JavaScript base

Competenze

![[Markdown]](https://img.shields.io/badge/Markdown-documentation-lightgrey)
![[Git]](https://img.shields.io/badge/Git-versioning-red)
![[GitHub]](https://img.shields.io/badge/GitHub-repositories-black)
![[HTML]](https://img.shields.io/badge/HTML5-base-orange)
![[CSS]](https://img.shields.io/badge/CSS3-base-blue)



Esempio Profile README

Qualche esempio:

- <https://github.com/2black0>
- <https://github.com/KrishCodesw>
- <https://github.com/Aj-Networks>
- <https://github.com/rmdip>
- <https://github.com/HamiltonPDev>

CV in Markdown pubblicato con GitHub Pages

Obiettivo

Scrivere un CV semplice usando Markdown.

Il CV deve essere:

- leggibile come file `.md`;
- versionato con Git;
- pubblicabile online;
- esportabile in PDF.

File richiesti

repository-dir/

└─ cv.md

└─ style.css

└─ README.md

└─ .github/

└─ workflows/

└─ publish.yml

CV in Markdown pubblicato con GitHub Pages

Template `cv.md`

Nome Cognome

****Ruolo / Profilo sintetico****

Breve descrizione in 2-3 righe.

Contatti

- Email: nome@example.com
- GitHub: <https://github.com/username>
- LinkedIn: <https://linkedin.com/in/username>
- Portfolio: <https://username.github.io>

Competenze

- Markdown e documentazione tecnica
- Git e GitHub
- HTML e CSS base
- JavaScript base
- Scrittura README

Esperienza / Progetti

Progetto FISM 2026

Repository documentata realizzata durante il corso.

- Creato README tecnico.
- Organizzata documentazione in `docs/`.
- Pubblicata pagina progetto con GitHub Pages.

Formazione

Corso FISM 2026

Argomenti principali:

- terminale;
- Git e GitHub;
- Markdown;
- documentazione tecnica;
- HTML/CSS;
- JavaScript base.

Lingue

- Italiano: madrelingua
- Inglese: livello ...

CV in Markdown pubblicato con GitHub Pages

CSS semplice per il CV - style.css

Serve a rendere il markdown più carino

Il contenuto deve restare in Markdown.

Il CSS serve solo a migliorare la resa visiva.

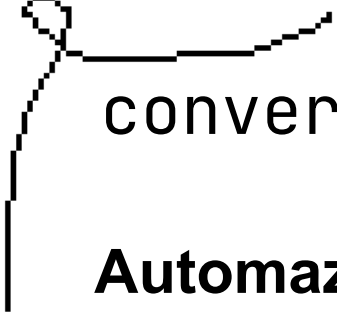
```
body {  
  max-width: 850px;  
  margin: 40px auto;  
  padding: 0 24px;  
  font-family: Arial, sans-serif;  
  line-height: 1.6;  
  color: #222;  
}
```

```
h1 {  
  border-bottom: 2px solid #222;  
  padding-bottom: 8px;  
}
```

```
h2 {  
  margin-top: 32px;  
  border-bottom: 1px solid #ddd;  
  padding-bottom: 4px;  
}
```

```
a {  
  color: #0366d6;  
}
```

```
code {  
  background: #f3f3f3;  
  padding: 2px 4px;  
  border-radius: 4px;  
}
```



convertire Markdown in HTML e PDF

Automazione con GitHub Actions

GitHub Actions usa workflow scritti in YAML. La documentazione ufficiale descrive un workflow come un processo automatizzato configurabile, composto da uno o più job, definito in un file YAML.

Per convertire Markdown in PDF possiamo usare Pandoc.

Pandoc è un convertitore universale che supporta molti formati, incluso Markdown, HTML, LaTeX, DOCX e PDF.

Ogni volta che facciamo push su `main`:

1. GitHub scarica la repository;
2. installa Pandoc e LaTeX;
3. converte `cv.md` in `index.html`;
4. converte `cv.md` in `cv.pdf`;
5. pubblica entrambi con GitHub Pages.



convertire Markdown in HTML e PDF



Workflow `publish.yml`

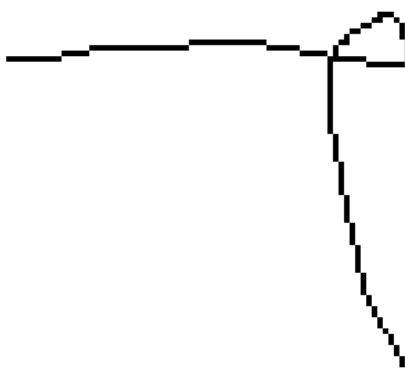
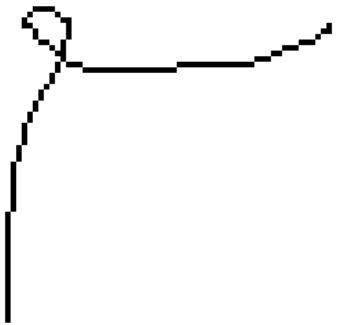
File `.github/workflows/publish.yml`

Questo ve lo fornisco io!

Prima di eseguire

Nel repository:

1. andare su **Settings**;
2. aprire **Pages**;
3. impostare la sorgente su **GitHub Actions**.



FINE



— WWW.DRACO.ME —